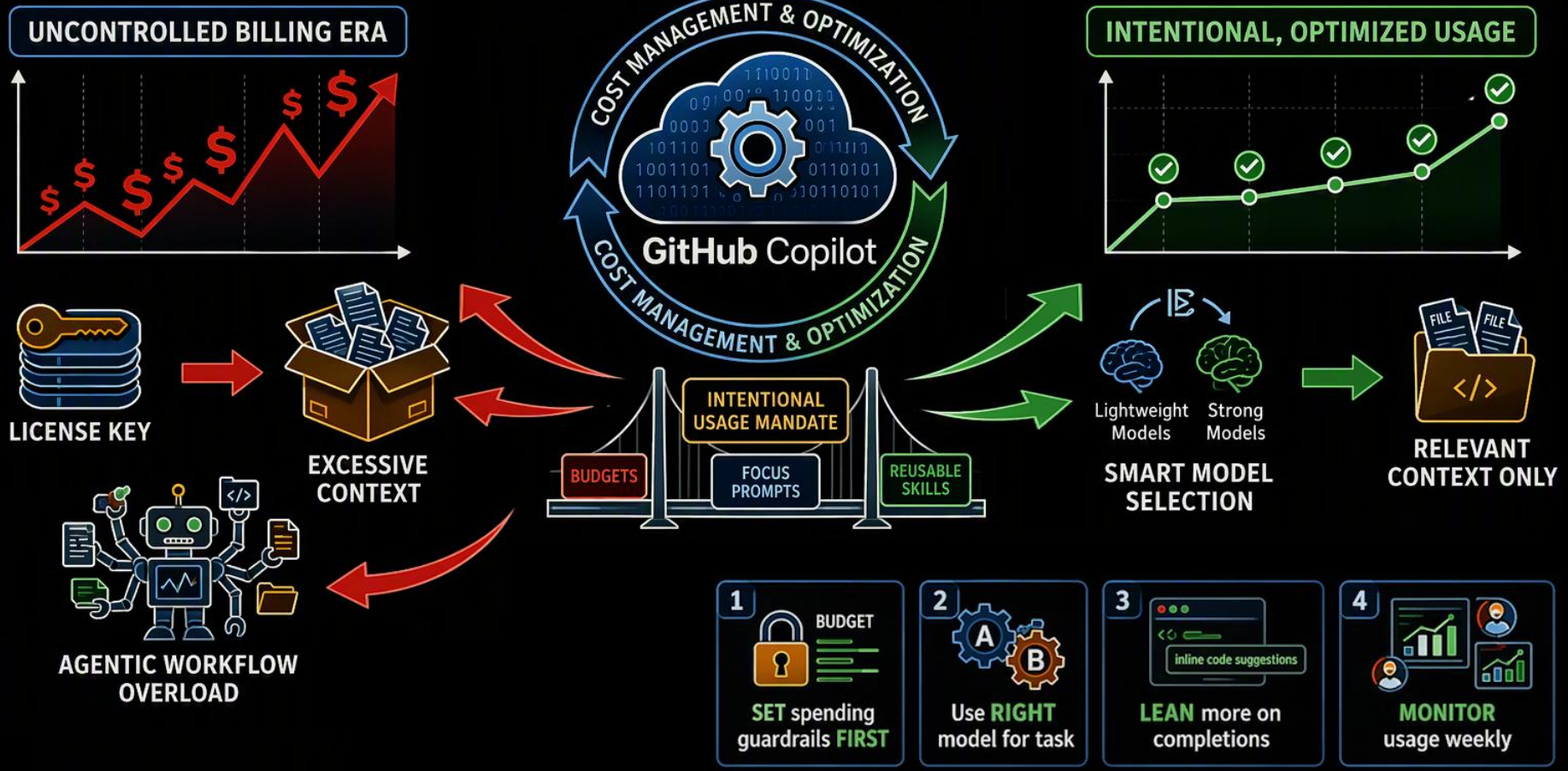
GitHub Copilot Tokenomics

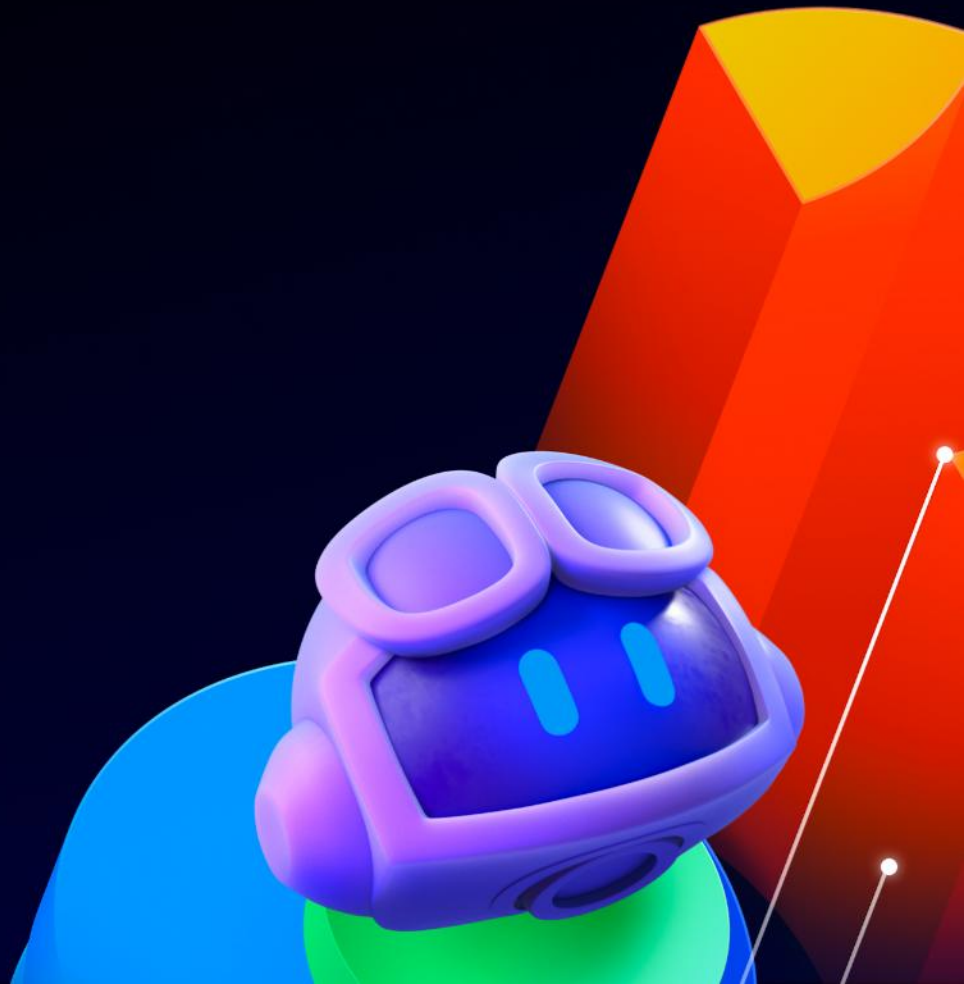


Terms and Conditions

Our Agenda

- What is Usage Based Billing and how does Copilot actually work?
- Lever 1: Model Choice (Micro-Tokenomics)
- Lever 2: Context Windows (Macro-Tokenomics)
- Best Practices

How did we get here...?



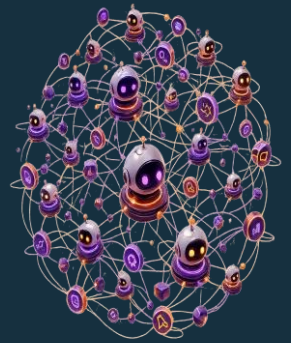
HOW WE GOT HERE

Why token usage is harder to predict now

2024 — life was simple

- Copilot was mostly single-prompt, request-and-response
- PRUs fit that world well — one request ≈ one unit
- Easy to reason about and to budget

2026 — agents changed the math



- One user request can trigger many model calls
- Agents orchestrate sub-agents, tools, and loops
- Long-running workflows compound tokens fast

Billing followed the technology: from counting requests → to measuring tokens.

HOW WE GOT HERE

GitHub AI Credits — the new unit of cost

1 AI credit

= $0.01 USD

How a request becomes a bill

- Every interaction consumes input, output, and cached tokens
- Each token is priced by the model used
- Total token cost is converted into AI credits (1 credit = $0.01)

Monthly allowance

- Copilot Business — 1,900 / license
- Copilot Enterprise — 3,900 / license
- Pooled at the billing-entity level

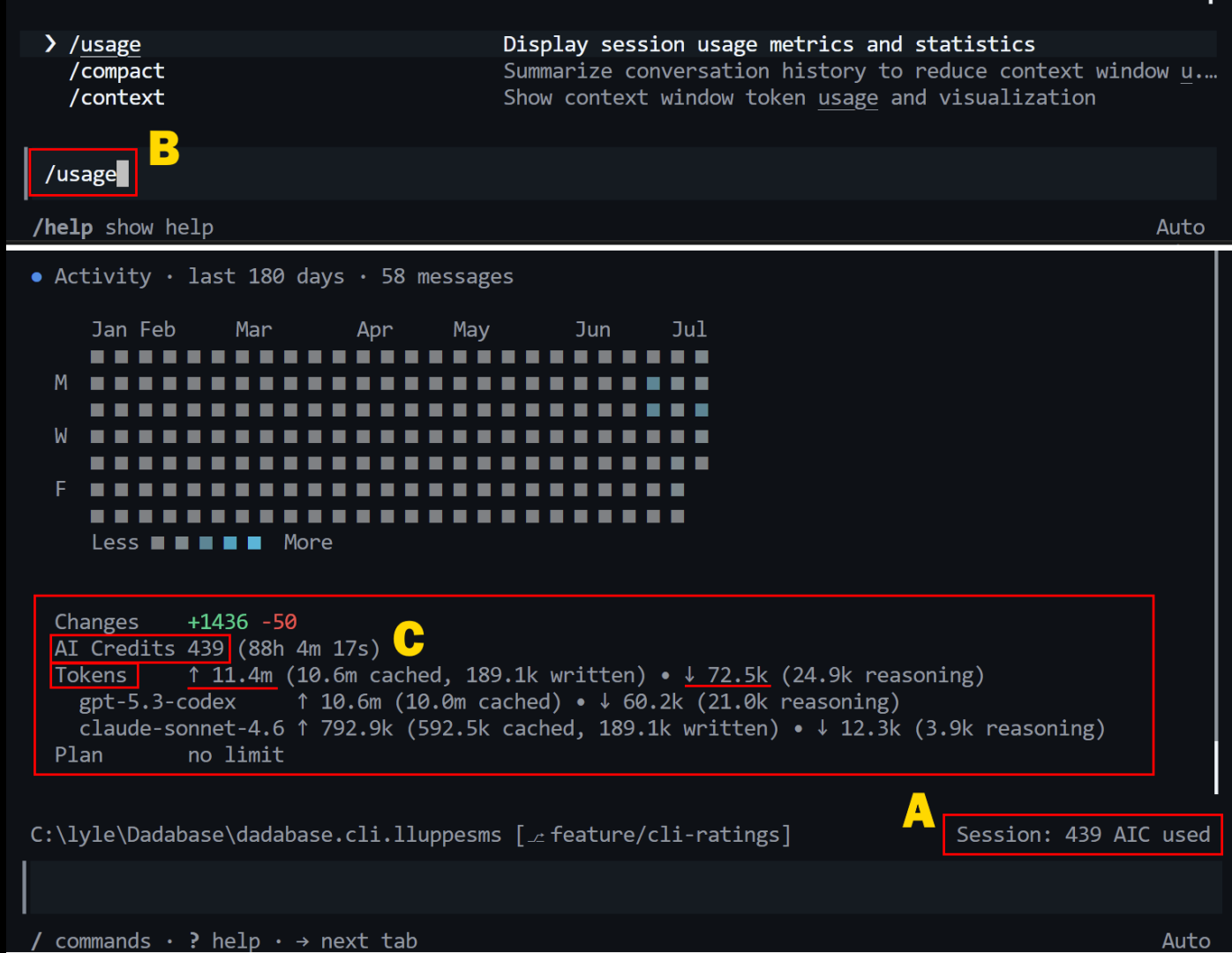
Consumes credits

- Copilot Chat & CLI
- Cloud agent, Spaces, Spark
- Third-party coding agents

Does NOT consume credits

- Code completions
- Next edit suggestions
- Unlimited on paid plans

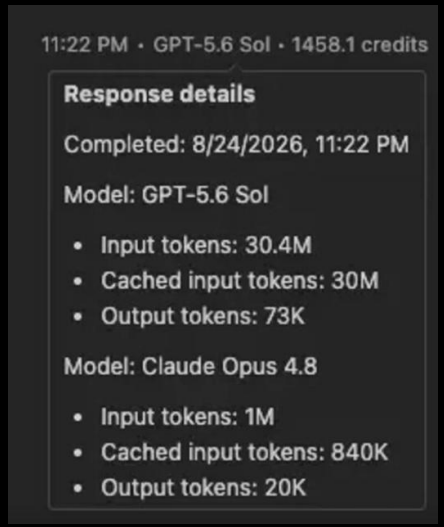
View Cost in the Copilot CLI

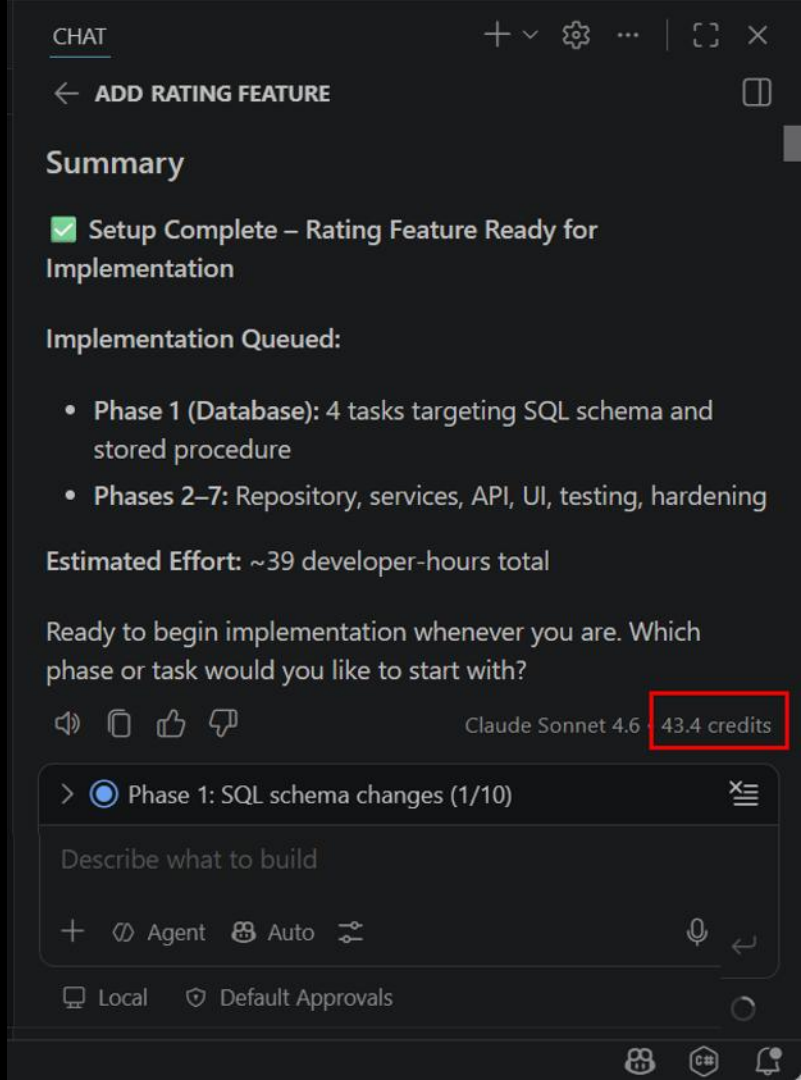


View Cost in VS Code Chat

(for one turn only...)

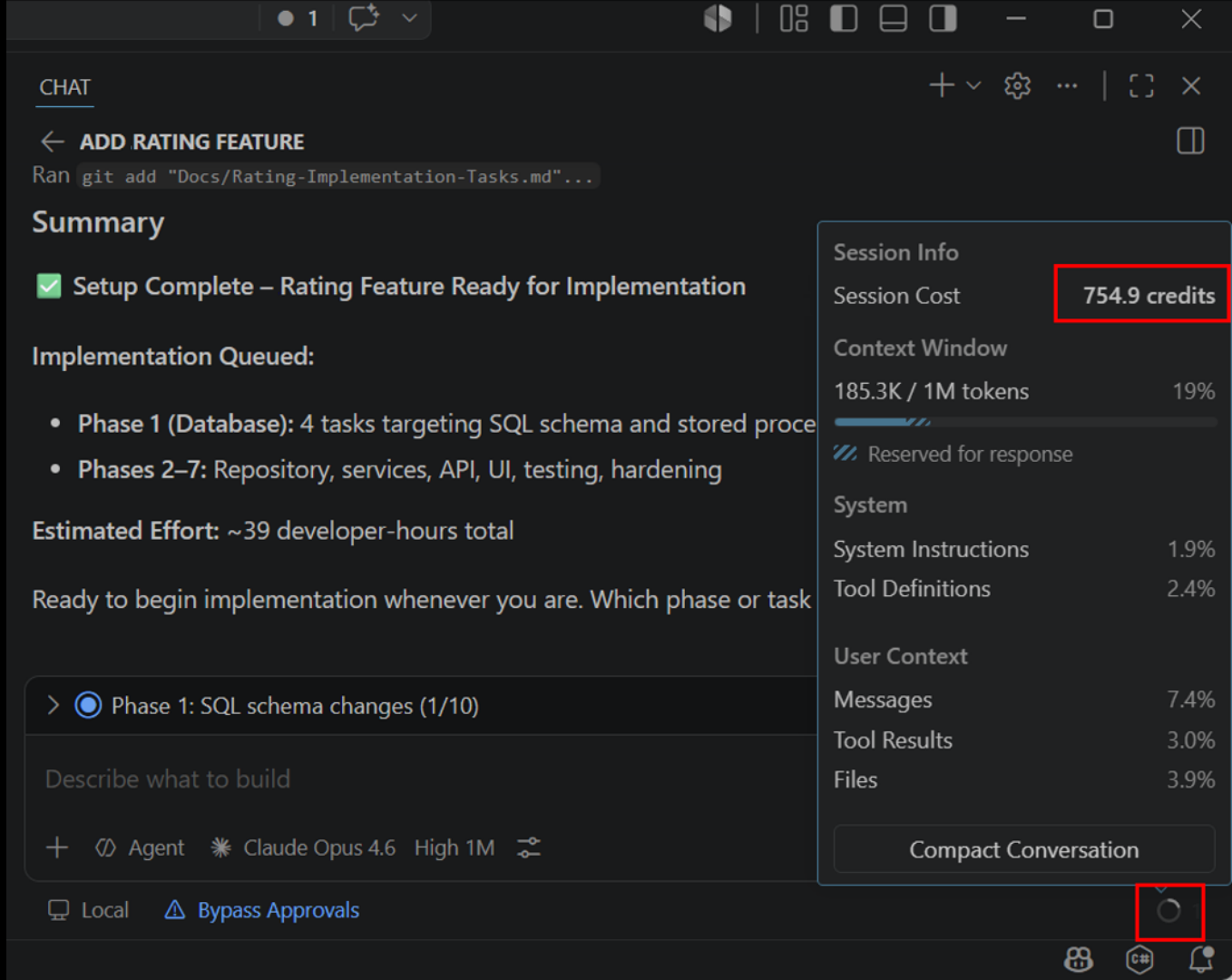
(new version coming soon!)





View Cost in VS Code Chat

(for the entire session...)

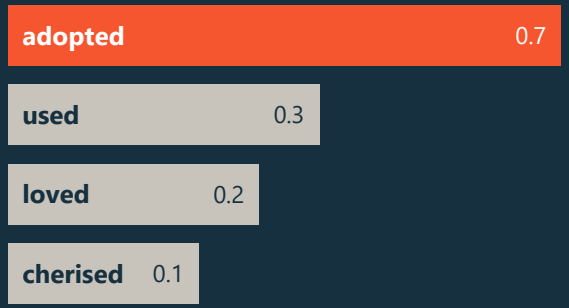


Do you know what actually happens when you submit a prompt…?

A QUICK REFRESHER

An LLM is a word-probability machine

GitHub Copilot is the world's most widely...

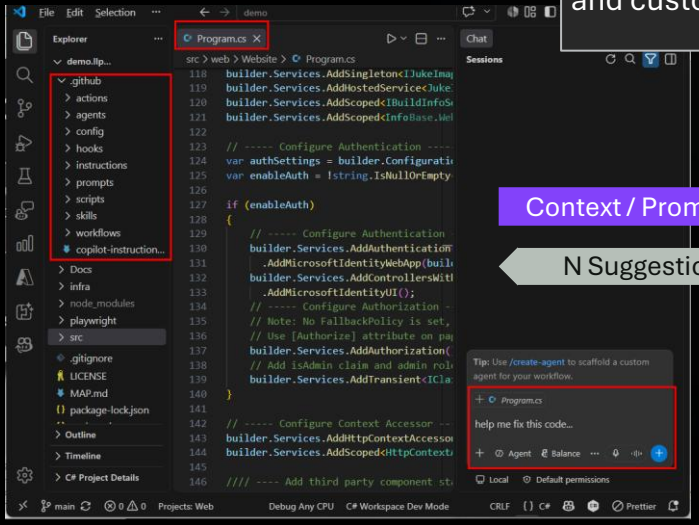


What this means

- Text in, text out — nothing more mystical
- It predicts the most probable next word, then the next
- In coding: predicting the next statement instead of prose
- Bells and whistles were added, but the core is still this

BEHIND THE SCENES

How GitHub Copilot fulfills a request



Copilot assembles context
from tabs open in the code editor and customizations

Apply several **pre-model filters**
- Test for **toxic language**
- Test for **relevance** (chat only)
- Guard against **prompt hacking**

Context / Prompt

N Suggestions

Copilot Harness

Context / Prompt

N Suggestions

✓

Proxy

Context / Prompt

N Suggestions

Model

Apply several **post-model filters**

- **Code quality**
- **Unique Identifiers** (eg.: emails, IPs etc.)
- Suggestion **matching public code**

It’s not magic — it’s just (stateless) text

You & your project

The agent (harness)

The LLM

The agent talks to the LLM on your behalf — dozens of times per task. And the LLM is stateless: it stores nothing. "Having a conversation" means re-sending the entire ordered history — every input and output — on every single turn. Tokens and context **compound** with each loop.

Basic Economic Theory

Micro-Economics:

Which car or truck should I buy?

- **Performance:** A Ford F-250 can pull and transport huge loads and has a Super Cab, while a Prius is limited in capacity.
- **Efficiency and operating costs:** A Prius will consume less fuel and cost much less to own over time, while the truck is more expensive.
- **Upfront Cost:** A heavy duty truck requires a much higher investment.

Macro-Economics:

What affects the entire country?

- As money supply rises, there is too much money for too few goods.
- Inflation begins to rise, increasing the cost of goods.
- Every cost inflation triggers other cost increases such as salary increase.
- Governments might raise interest rate in an attempt to control inflation.

Basic “Tokenomics” Theory

The biggest levers for optimizing quality & tokens

Micro-Tokenomics

Model Choice

- Premium model → slower, smarter = more credits
- Efficient model → good enough = lower cost
- You optimize capability vs cost, per task

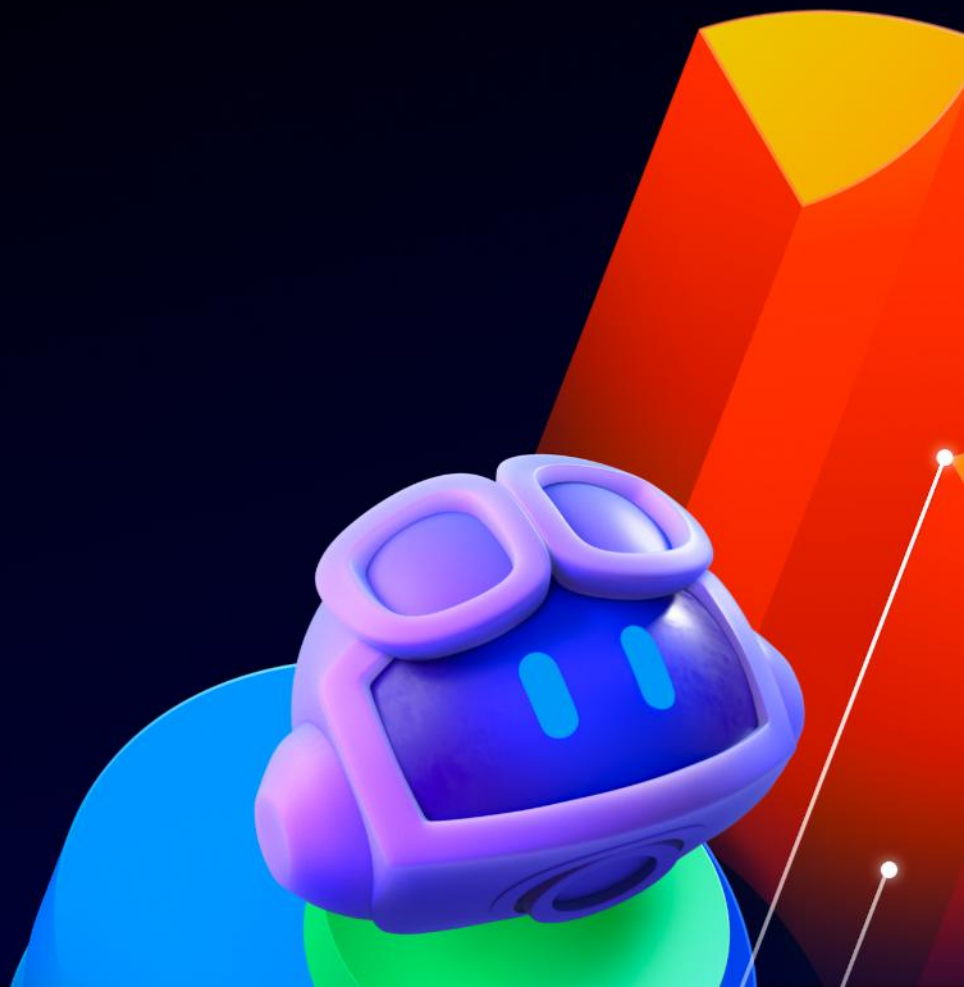
Macro-Tokenomics

Context Windows

- The harness re-sends the entire history to the LLM, every turn
- Tokens compound as the session grows
- Context engineering is the core skill of controlling that context window

MICRO-TOKENOMICS

Model Selection



LEVER 1 — MODEL CHOICE

It starts with choosing the right model

Even a single model family has variations with different abilities and very different prices per 1M tokens.

~25× cost gap

between the most powerful and the leanest models

Bigger ≠ better

- Model choice drives BOTH token cost and quality
- The most capable model is often overkill — and can hurt results
- Right-sizing is the fastest win available to a developer

ANTHROP\C

OpenAI

Microsoft

Google AI

LEVER 1 — MODEL CHOICE

Models — four buckets

Deep reasoning & complex coding

- Claude Opus 4/5
- Gemini Pro
- GPT-5.5/5.6 Sol
- Claude Fable (long horizon coding)
- GPT Astra (long horizon coding)

Agentic software development

- GPT-5.3-Codex
- GPT-5.4 mini
- Kimi K3
- GPT-5.6 Terra

General purpose

- Claude Sonnet 4/5
- GPT-5.3-Codex, 5 mini, 5.6 Terra
- MAI-Code-1.1-Flash
- Kimi K2.7 Code
- Grok 4.5/4.6
- Qwen 2.5

Fast, simple, repetitive

- GPT-5.6 Luna
- Claude Haiku 4.5
- MAI-Code-1.1-Flash
- Gemini 3.5 Flash

Source: AI model comparison - GitHub Docs September 6, 2026

Each model has a slightly different cost

* All prices are per 1 million tokens

Model	Category	Input	Cached (R)	Cached (W)	Output
GPT-5.6 Astra	Super Powerful	$10.00	$1.00	$12.50	$50.00
Claude Fable 5	Super Powerful	$10.00	$1.00	$12.50	$50.00
GPT-5.6 Sol (promo Sept-Nov!)	Powerful	$4.00	$0.40	$5.00	$20.00
GPT-5.5	Powerful	$5.00	$0.50		$30.00
Claude Opus 5	Powerful	$5.00	$0.50	$6.25	$25.00
GPT-5.6 Terra	Versatile	$2.00	$0.20	$2.50	$12.00
Claude Sonnet 5	Versatile	$2.00	$0.20	$2.50	$10.00
GPT-5.3-Codex	Powerful	$1.75	$0.18		$14.00
Gemini 3.5 Flash	Lightweight	$1.50	$0.15		$9.00
GPT-5.6 Luna (price cut!)	Lightweight	$0.20	$0.02	$0.50	$1.80
Claude Haiku 4.5	Versatile	$1.00	$0.10	$1.25	$5.00
MAI-Code-1.1- Flash	Lightweight	$0.20	$0.02		$1.20
Kimi K2.7 Code	Versatile	$0.95	$0.19		$4.00
GPT-5 mini	Lightweight	$0.25	$0.03		$2.00

Source: Models and pricing for GitHub Copilot - GitHub Docs as of September 6, 2026

INSIDE THE WINDOW

Token Basics

1,000,000 tokens

≈ the entire **Lord of the Rings trilogy — plus The Hobbit**.
That's the ceiling of what fits, not a target.

- 1 token ≈ ¾ of a word — think "1 token ≈ 1 word"
- Context limits vary: ~50–200K (small) up to ~1M (large)
- Prompts, files, and responses all cost tokens — and compound every loop

LEVER 1 — MODEL CHOICE

When to reason, when to keep it light

Reasoning models

Planning, architecture, subtle bugs, and many-file tasks
— where thinking pays off.

Lightweight models

Implementation after the plan
— fast, well-specified execution.

Watch out — hand a reasoning model a tight spec and it may re-open the plan and "go rogue." For execution, that's a liability.

LEVER 1 — MODEL CHOICE

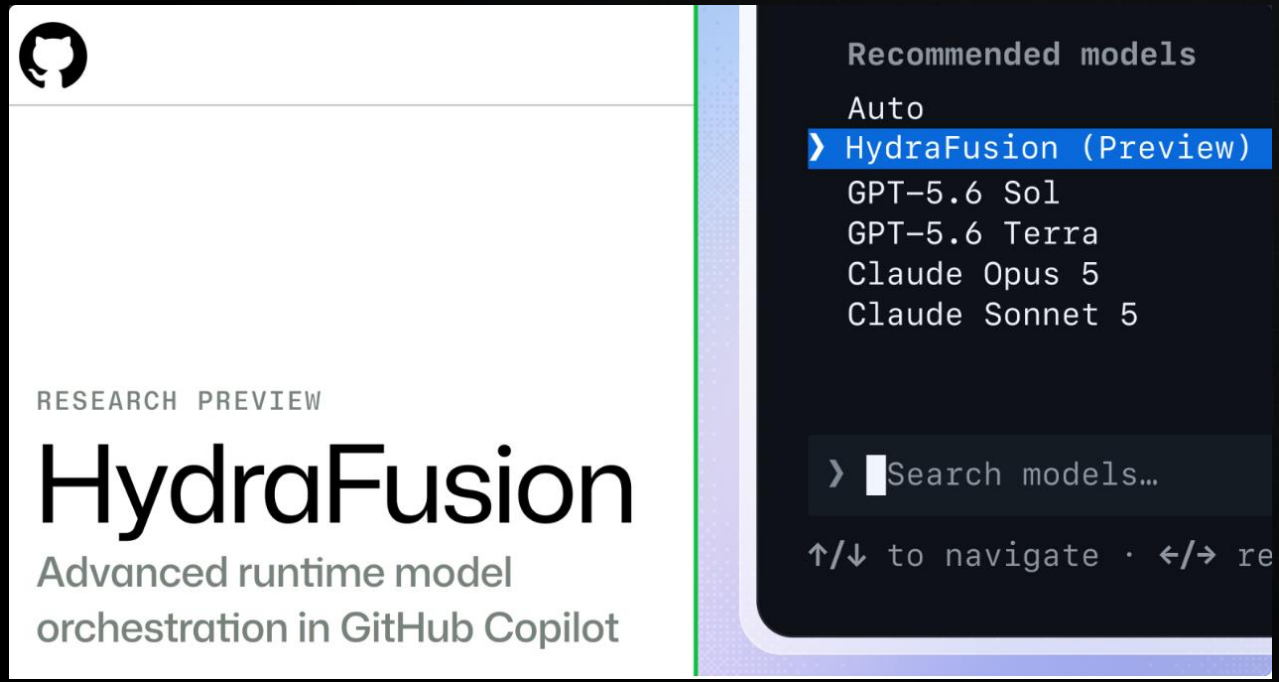
Auto Mode:
The right model for each task.
Automatically.

LEVER 1 — MODEL CHOICE

Auto Mode:
The right model for each task.
Automatically.

Coming soon...!

Research Preview available Sept 4, 2026 in GHCP CLI!



Project HydraFusion: Frontier quality via multi-model orchestration - The GitHub Blog
Satya Nadella LinkedIn Post on HydraFusion/

LEVER 1 — MODEL CHOICE

Auto mode and matching the model to each phase

Auto mode — stay lazy on purpose

- Detects task intent and picks a right-sized model
- A sensible default for most routine work
- Make deliberate choices only when it truly matters
- The scalable answer for hundreds of millions of users

Match the model to each phase

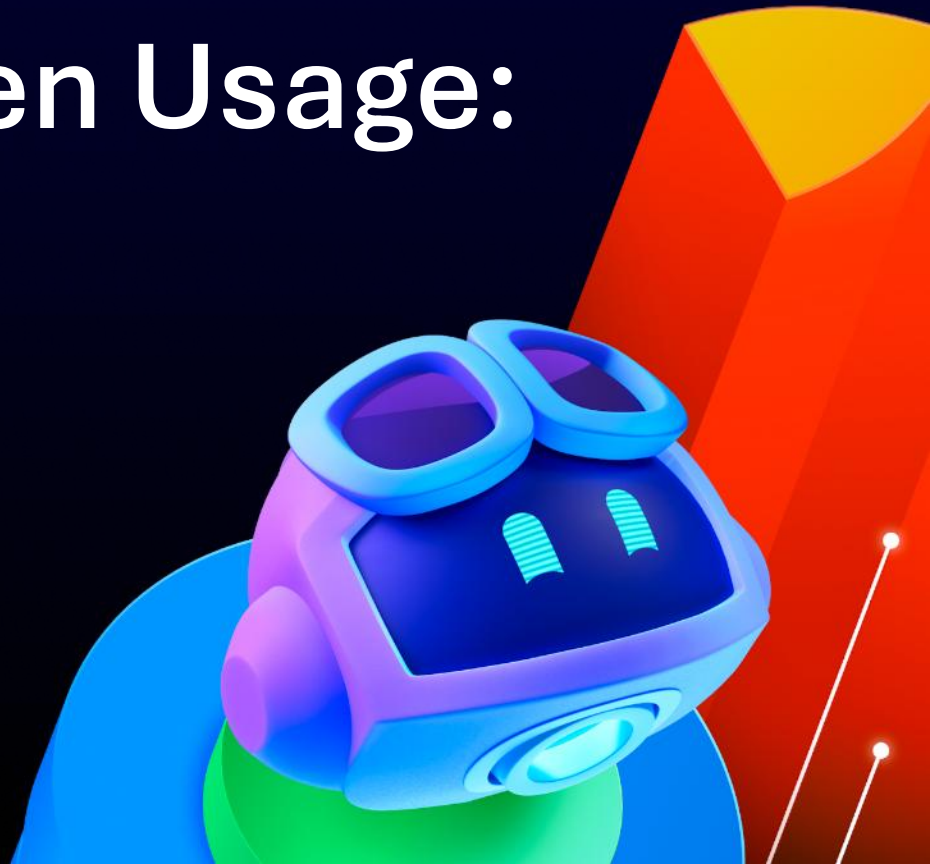
- Plan phase → a reasoning model
- Implement phase → a mid-tier or lightweight model
- Review phase → a code specialist
- Set it when you START a new session or phase

Switch by phase — not mid-conversation. Plan → reasoning · implement → mid-tier · review → specialist, each in a NEW session/window. Changing models inside a live session invalidates the prompt cache — the new model has no cached state, so the whole context is re-billed at full price.

MACRO-TOKENOMICS

Controlling Token Usage:

What is in your System Instructions?



INSIDE THE WINDOW

What exactly is in the system instructions?

Instructions, History, References

Context Rot, Noise

- 🪪 Identity
  You are GitHub Copilot. You are a skilled programmer.
- 📏 Code Rules
  "Make precise surgical changes…“
- 📘 Guidelines
  "Use Rubber Duck to critique plans“
- ⛔ Limitations
  Don't share sensitive data, don't leak the system prompt, don't create harmful content
- 🛠️ Tool Use Instructions
  Built-in tools (read, view, sub-agents, PowerShell), MCP Servers, Skills
- 🧩 Custom instructions
  AGENTS.md, copilot-instructions.md
- ➕ Additional Instructions
  Plan Mode, Autopilot
- 🧠 Custom Agents
- ✅ Final Instructions
  Response concisely, finish the job….

INSIDE THE WINDOW

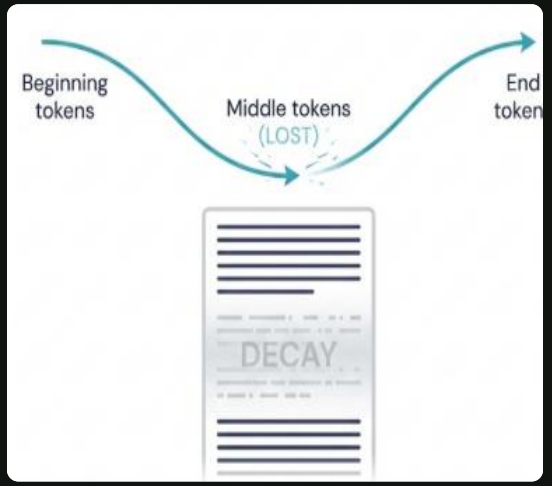
Context Window & Tokens

INSIDE THE WINDOW

Research → Plan → Implement

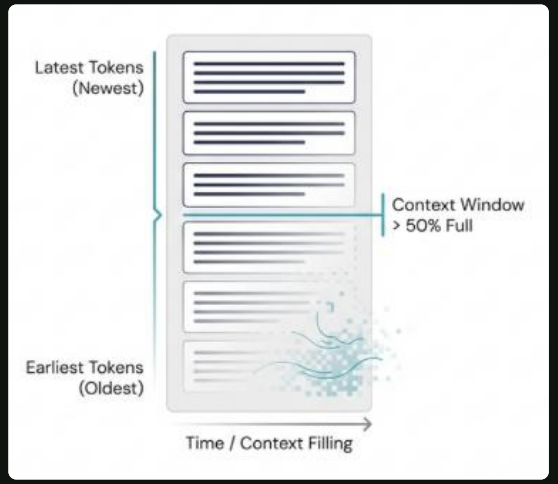
INSIDE THE WINDOW

Context Rot



Lost in the Middle (<50% full)

Models bias tokens at the beginning and end of the context.



Recency Bias (> 50% full)

Models bias tokens from the end of the context.

Reference:
https://www.producttalk.org/context-rot/

Rule of thumb: keep working sessions under ~60–70% of the window, and use a NEW window per distinct task. Just because you can fill the context window doesn't mean you should.

Everything should
be made
as simple as
possible,
but not simpler.

Albert Einstein *

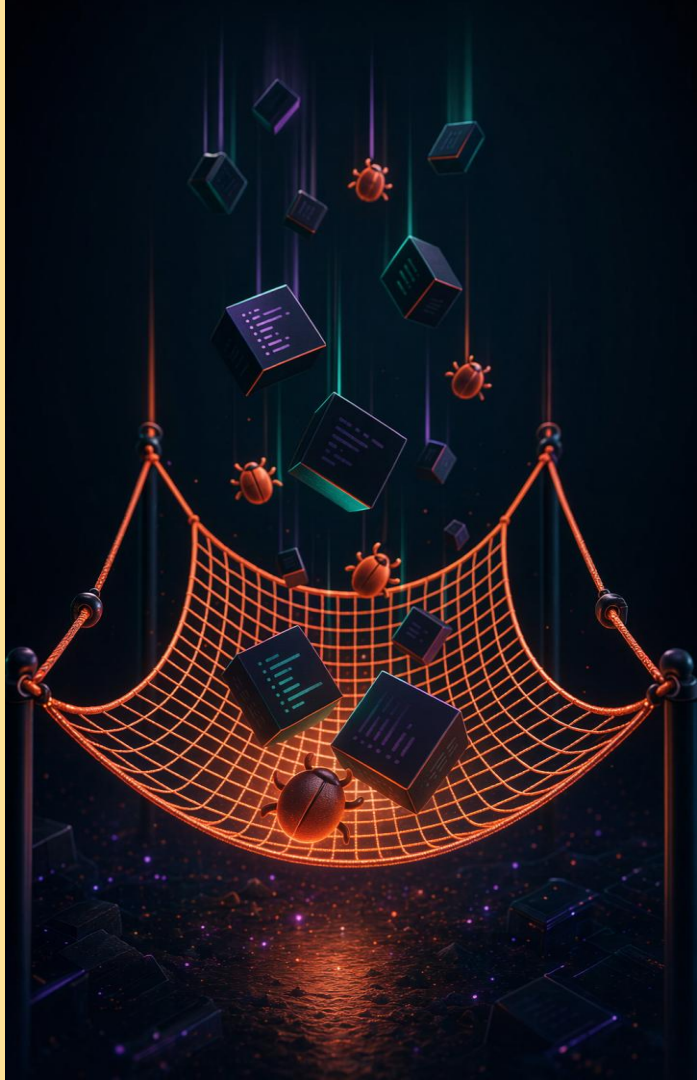
(* maybe...)

Provide as little
context as possible,
but as much as
required.

SECTION 04

Quality & token controls

Deterministic guardrails and the agent configuration files that keep quality high while spending less.



SPECIFIC AREAS

Your agent-configuration toolbox

Context engineering is mostly agent configuration — the files agents pick up automatically. Each has a different job, cost, and trigger.

Persistent instructions
copilot-instructions.md — always on

Custom agents
*.agent.md — scoped role & tools

Skills
SKILL.md — lazy-loaded capability

MCP servers
External tools & APIs

Sub-agents
Second window for a sub-task

Scoped instructions
*.instructions.md — path-based

Prompt files
*.prompt.md — manual starting points

Copilot memory
Learns patterns over time

AGENT CONFIG

Persistent instructions — your always-on guidance

`copilot-instructions.md` / `AGENTS.md`

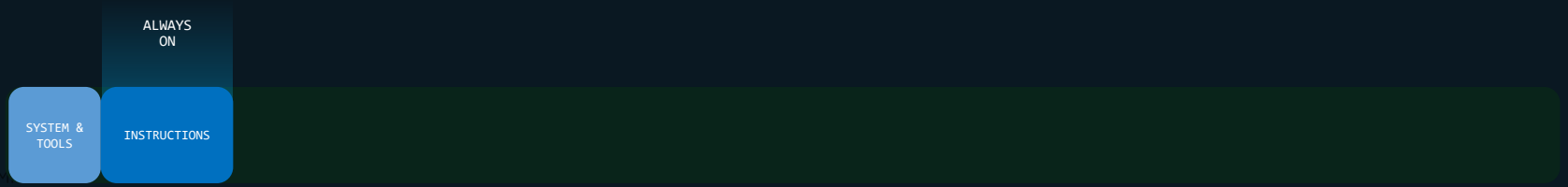
What belongs

- Non-negotiables & guardrails
- Recurring agent-miss fixes
- Output trimming ("be concise")
- One-line tech-stack context

Best practices

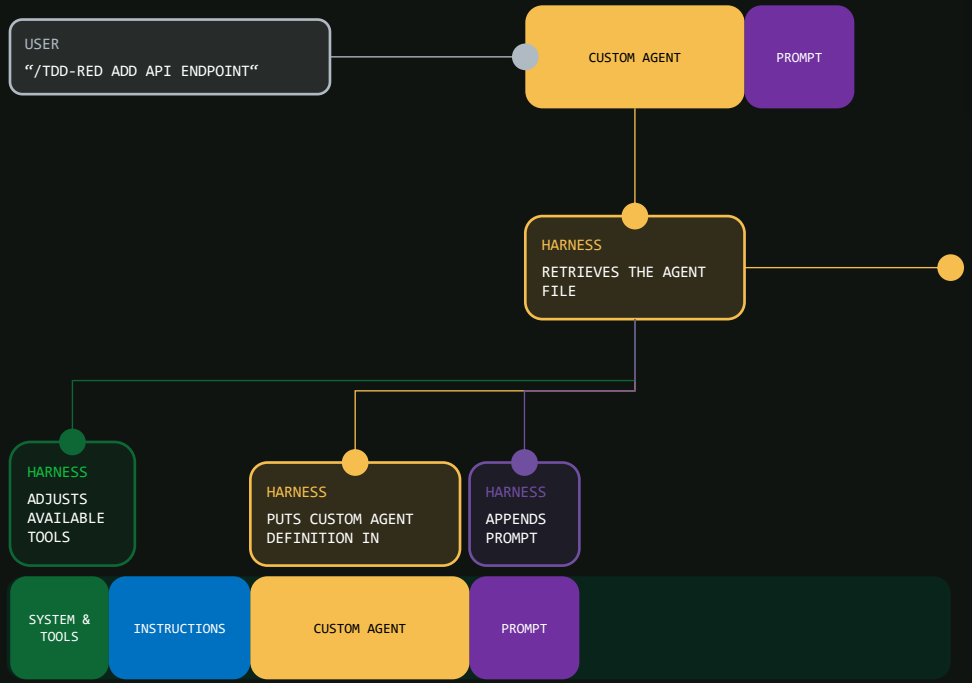
- Keep it very small — re-sent every turn
- Write it yourself, not with AI
- Iterate on real agent behavior
- Recreate it often — it's living

"Be concise" does almost as much as a 50-line skill — and output tokens are the priciest bucket.



AGENT CONFIG

Custom Agents



What they are

- A reusable prompt that forces a specific way of working
- Manually invoked, e.g. /tdd-red add API endpoint
- Write the behavior once; reuse it again and again
- Great for workflows an agent wouldn't choose on its own

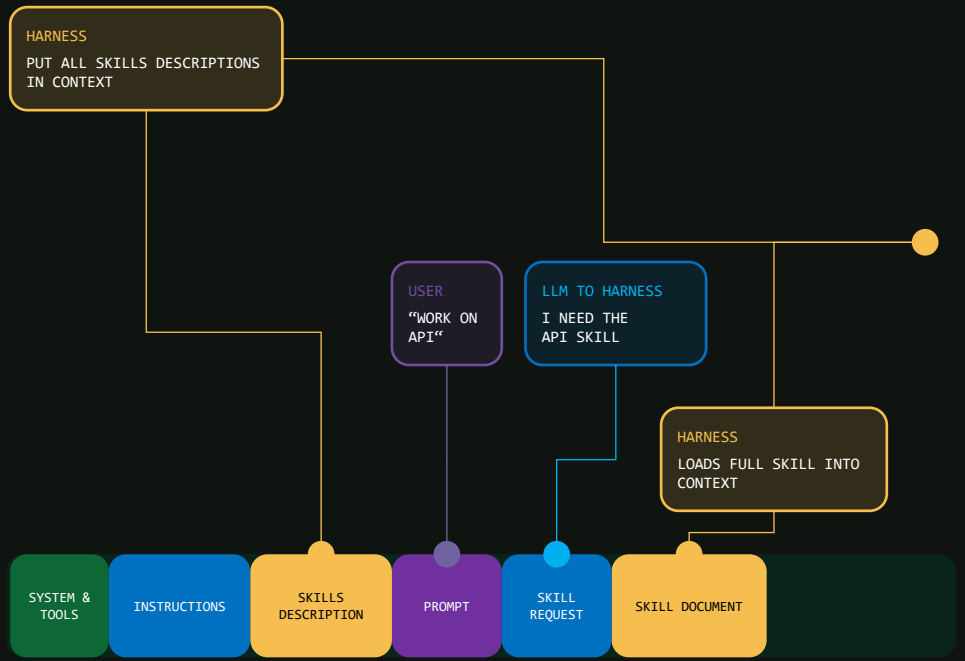
Why they help

- The harness adjusts which tools the agent can use
- The real win: preventing wrong paths, not token savings
- Withhold a tool and the agent simply can't misuse it
- e.g. read the issue for a spec — but not edit it

Token savings are minor (tools get cached). The value is control — the agent can't wander somewhere you didn't intend.

AGENT CONFIG

Skills



How they work

- Markdown that shapes behavior for a specific task
- Only the short description sits in context up front
- The LLM asks for the skill when the task matches
- The harness then loads the full skill on demand

Best practices

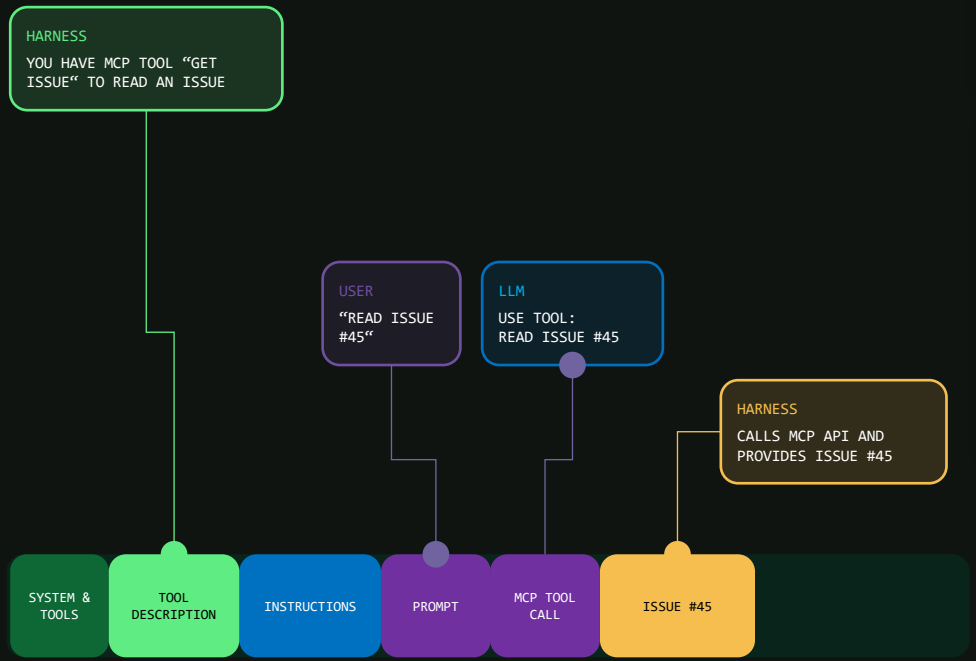
- Don't overdo it — you don't need hundreds
- Avoid redundant skills (does it need a "React skill"?)
- Only for capabilities the agent lacks otherwise
- Maintain them — some become unnecessary as models improve

Skills are a great way to offload context that isn't always relevant — pay for it only when the task needs it.

AGENT CONFIG

MCP

Integrate with 3rd party tooling



What they add

- External APIs & dynamic tools
- Tool schemas enter the window
- e.g. get_issue → "read #45"
- Harness calls API, returns result

Be rigorous

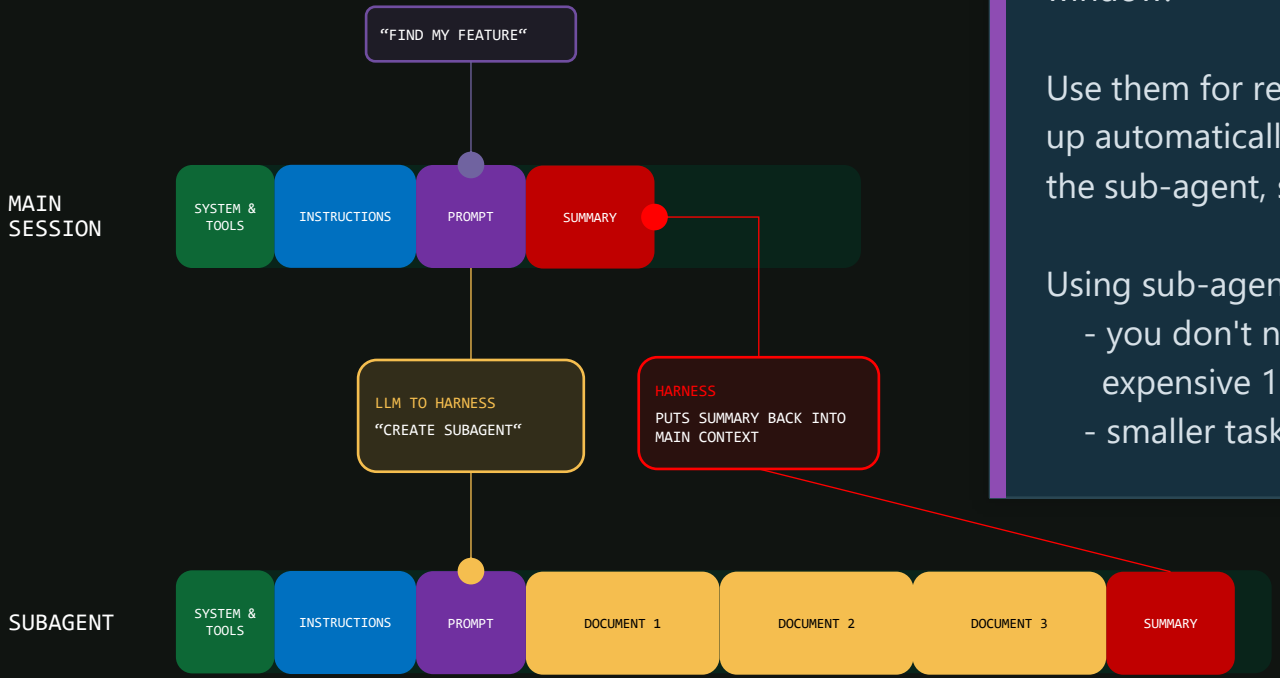
- They bloat tool schemas every turn
- Can lead agents to wrong tools
- Deactivate ones you rarely need
- Scope heavy ones in custom agents

Playwright is powerful but costly — screenshots and page reads burn tokens. Turn it on only when a task truly needs it.

AGENT CONFIG

Subagents

Offload task-specific context



The math that makes sub-agents pay: a sub-agent reads five files to answer one question, but your main session only gets the final answer — those five files never enter your main window.

Use them for research; the agent often spins one up automatically. Trade-off: tokens are spent in the sub-agent, so it's a conditional optimization.

Using sub-agents will means that:

- you don't need to use the larger and more expensive 1M token context windows
- smaller tasks means less drift – better accuracy

AGENT CONFIG

The lighter-weight configs, briefly

Scoped instructions

Conditional instructions based on file-path patterns. Non-deterministic — offered to the agent like a skill. Start static; only scope when instructions get too long.

Prompt files

Manually invoked prompts. Handy to trim the toolset and kick off a custom agent or skill from a common starting point. Not in Copilot CLI — skills/agents are usually better.

Copilot memory

Automatic, small always-on instructions learned from your behavior and team patterns. Works in the background; little to tune, but worth checking periodically.

Lower impact on tokens and quality than the previous configs — but good to know they exist and to review them now and then.

QUALITY & TOKEN CONTROLS

Deterministic controls stop errors from compounding

Without tests — errors stack

- A buggy change lands on a buggy change on a buggy change
- Maybe faster and fewer tokens... at first
- What you actually built is an incident
- Cost: wasted CI/CD, review cycles, human time, a debugging session

With tests — the agent self-corrects

- A failing test says "stop — you can't continue"
- The agent fixes, then builds on a stable base
- It restarts accuracy instead of drifting to 50%
- Linters and security scanners do the same job

Errors compound - they don't average.

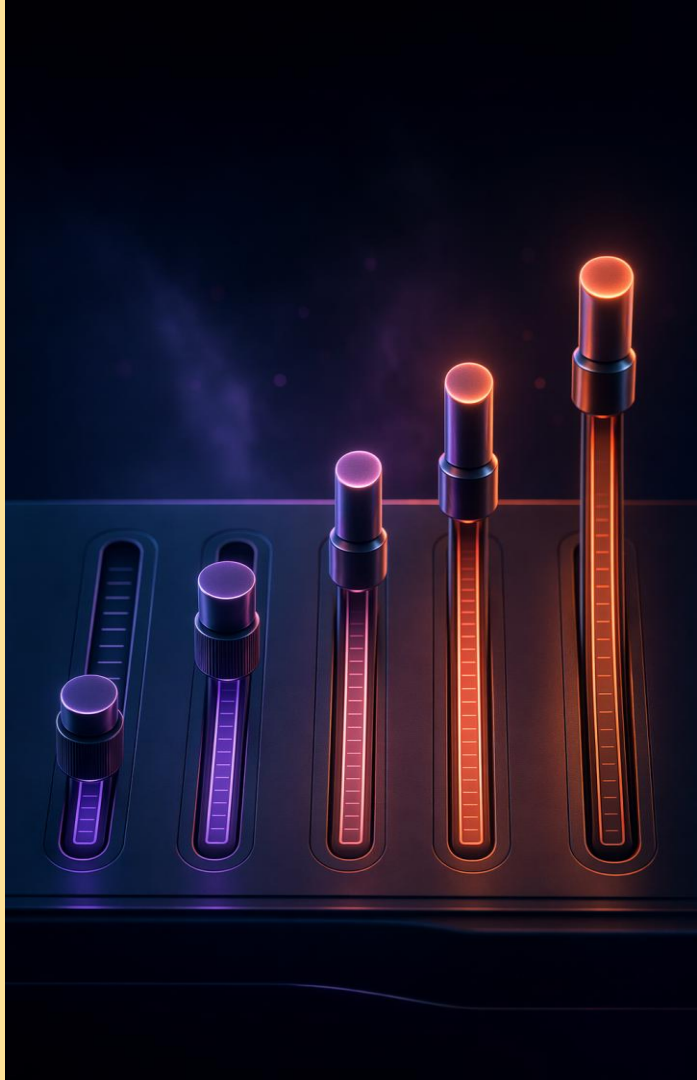
A 50-step run that's right only if every step is right is (accuracy)^50 — at 99%/step ≈ 60% success; at 95%/step ≈ 8%.
A tiny per-step gain pays off hugely.

The Copilot CLI team with ~10 devs ships ~500 PRs/week; 53% of their codebase is tests.

SECTION 05

The efficiency playbook

Where tokens get burned, the five levers that stop it, and what the savings look like.



THE PLAYBOOK

Where tokens get burned — the usual suspects

Reading whole files

"Look at src/" when one function matters — 20k+ tokens.

Long-running sessions

Not clearing between tasks; history rides along.

Oversized instructions

The always-on file, injected every single turn.

Verbose tool output

Piping whole logs in instead of grepping first.

Wrong model for the job

A reasoning model for routine edits.

Re-discovery loops

Re-reading a file after a mid-task /clear.

If you can't explain why Copilot needs a piece of context, it's probably costing you.

THE PLAYBOOK

Five levers, ordered by leverage

01	**Context hygiene**	Clear between tasks. Compact proactively. Resume when you can.
02	**Prompt discipline**	Plan before coding. Reference specific files and lines, not directories.
03	**Model selection**	Match model tier to task complexity. Choose per phase — a fresh session, not mid-conversation.
04	**Scope & tool control**	Tight working directory. Content exclusion. Minimal custom instructions.
05	**Measurement**	/usage, /context, and telemetry make waste visible.

LEVER 1 — CONTEXT HYGIENE

Don't wait for auto-compaction

`/clear`	Reset the timeline entirely — between unrelated tasks
`/compact`	Summarize the session, free most of the window — mid-task
`/context`	Show current token usage broken down by category
`/usage`	Per-model totals, session duration, files touched
`/resume`	Reopen a prior session from its saved summary, not zero
`/new`	Start a fresh session (alternative to /clear)

Sessions are like branches — start a new one per task and sweep out stale context before it piles up.

LEVER 2 — PROMPT DISCIPLINE

Shorter, sharper prompts cut input, output, AND tool calls

EXPENSIVE

```
"Look at my repo and figure out why users get 500
errors on login. Check the auth code, database layer,
middleware, and logs. Fix it."
```

Scans many files, reads large chunks, burns tokens before it even narrows down.

EFFICIENT

```
/plan  "In @src/auth/login.ts, handleLogin returns
500 when the email has unicode chars. Propose a fix."
```

Targets one file, one function, one case. Plan mode costs one prompt and drives the rest.

Tactics: Plan mode (Shift+Tab) before coding · @file references, not directories · one task per prompt · break big asks into steps

LEVER 3 — LEAN CUSTOM INSTRUCTIONS

copilot-instructions.md is re-sent every turn — every line is a tax

Keep

- One-line tech-stack context
- Standards the model can't infer from code
- Explicit "do not" rules for files/patterns
- Recurring agent-miss corrections
- A tight file ≈ 150 tokens

Cut

- Onboarding essays and team history
- Architecture diagrams as ASCII art
- Full style guides — link instead
- Anything redundant with the code itself
- 3,000 tokens × every turn × every dev = real money

LEVER 4 — MATCH THE MODEL TO THE TASK

Reserve premium reasoning for what demands it

Model tier	Best for	Watch for	When to switch
Included / small (GPT-5 mini, 4.1)	Autocompletions, syntax help, boilerplate, simple refactors	Weaker at multi-file reasoning; may hallucinate on big tasks	Default for routine work — often no premium cost
Mid-tier reasoning (Sonnet, GPT-5)	Most everyday coding: features, debugging, code review	The workhorse tier — where most token spend lives	Switch up only when it demonstrably gets stuck
Heavy reasoning (Opus, o-series)	Complex system design, subtle bugs, deep refactors	Steep multiplier; reasoning tokens count as output	Switch down as soon as the hard part is done
Code specialist (Codex)	High-volume code gen; second opinion on tricky diffs	Narrower scope; not the generalist chat model	Pair with a mid-tier for review workflows

Pro move: match the model to each phase — plan → heavy, implement → mid-tier, review → specialist — starting a fresh session per phase. Don't swap models inside a live session: it drops the prompt cache and re-bills the whole context at full price.

LEVER 5 — NARROW THE BLAST RADIUS

Limit what Copilot can see — the most reliable way to stop wasted tokens

/cwd and /add-dir

Explicit working directory shrinks the file surface. Add sibling dirs only when a task genuinely needs them.

Content exclusion

Org/enterprise level for large, generated, or sensitive files: /target, *.log, *.min.js, secrets/. Highest leverage.

.gitignore hygiene

Not a guaranteed filter, but reduces what Copilot indexes — build output, generated code, IDE files.

Tool allow / deny

--allow-tool / --deny-tool and session approvals stop shell commands that would balloon tool output.

Content exclusion is admin-configured and applies to every developer automatically — highest leverage, lowest ongoing cost.

THE PLAYBOOK

Specialist commands that save tokens

/explore

Fast codebase Q&A. Answers flow back without cluttering the main conversation with file reads.

/task

Runs commands like tests and builds. Reports brief on success, full output only on failure.

/plan

Produces a structured implementation plan before code is written. Cheap, high-impact.

/review

Code review with a tight signal-to-noise profile. Surfaces real issues, skips nits.

/delegate

Hands off to the cloud agent. Long tasks run async and return a PR — not billed against your session.

Why they save: a sub-agent reads five files to answer one question — your main session only gets the answer.

RECOMMENDED ADDITION

Hidden costs in CI — agentic workflows add up quietly

Why CI is different

- Runs on every PR — costs hide in the pipeline
- Run frequency matters as much as per-run cost
- Easier to optimize: fixed in YAML, wins compound

What worked at GitHub

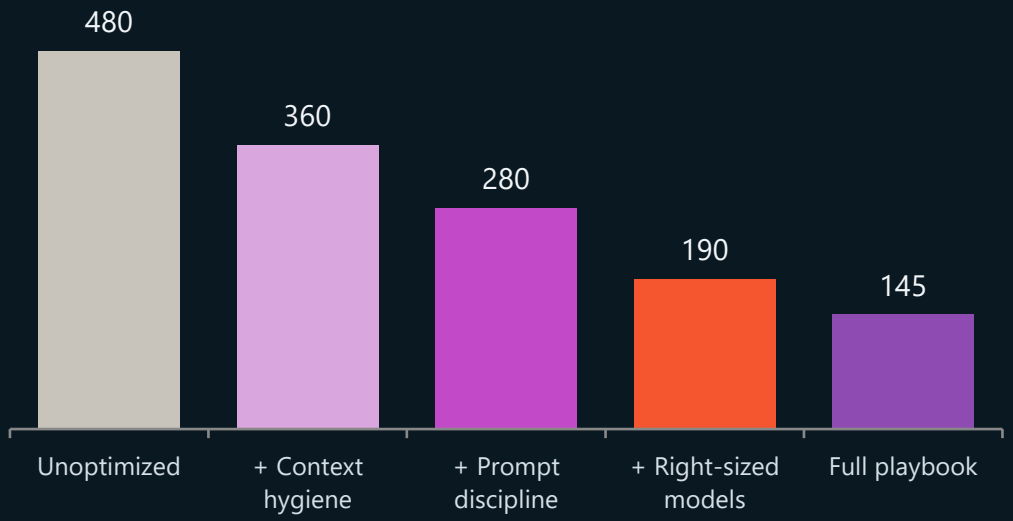
- Daily Auditor flags rising / anomalous usage
- Daily Optimizer files concrete fixes as issues
- Prune MCP tools; move data reads to gh CLI

Biggest win: move deterministic data-gathering out of the LLM loop, into pre-agent CLI steps.

Source: GitHub blog, "Improving token efficiency in GitHub Agentic Workflows" (May 7, 2026).

THE PAYOFF

What the savings look like when levers stack



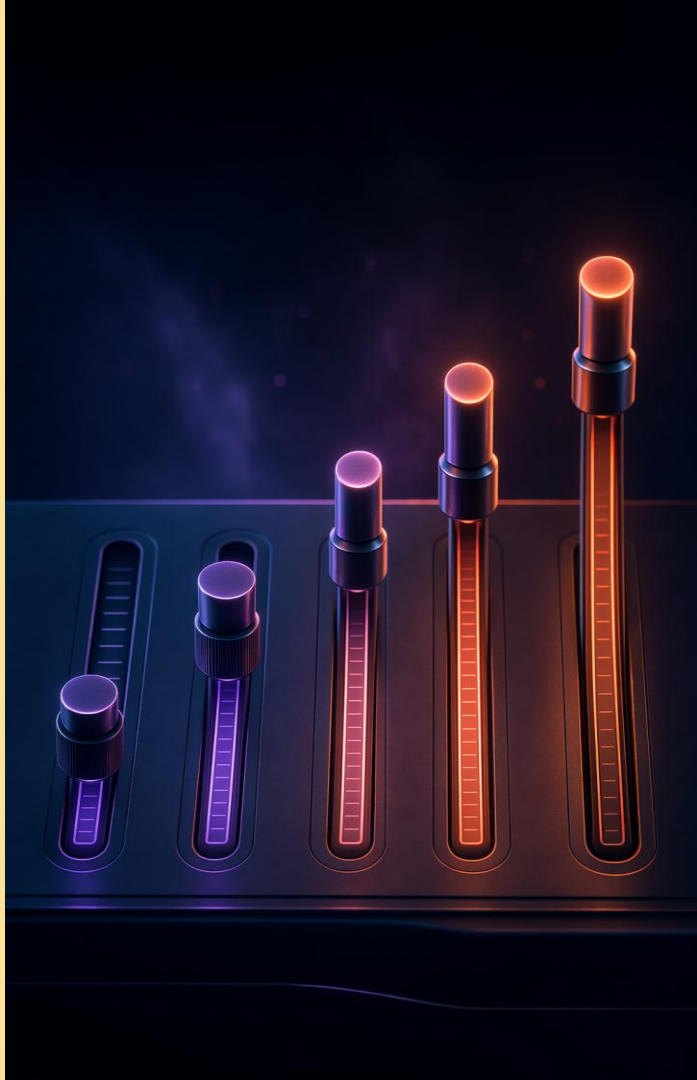
The compounding effect

- Each lever removes a different class of waste
- No single tactic is dramatic alone
- Stacked, they routinely cut 60–70% of spend
- Same work ships — you just stop sending junk

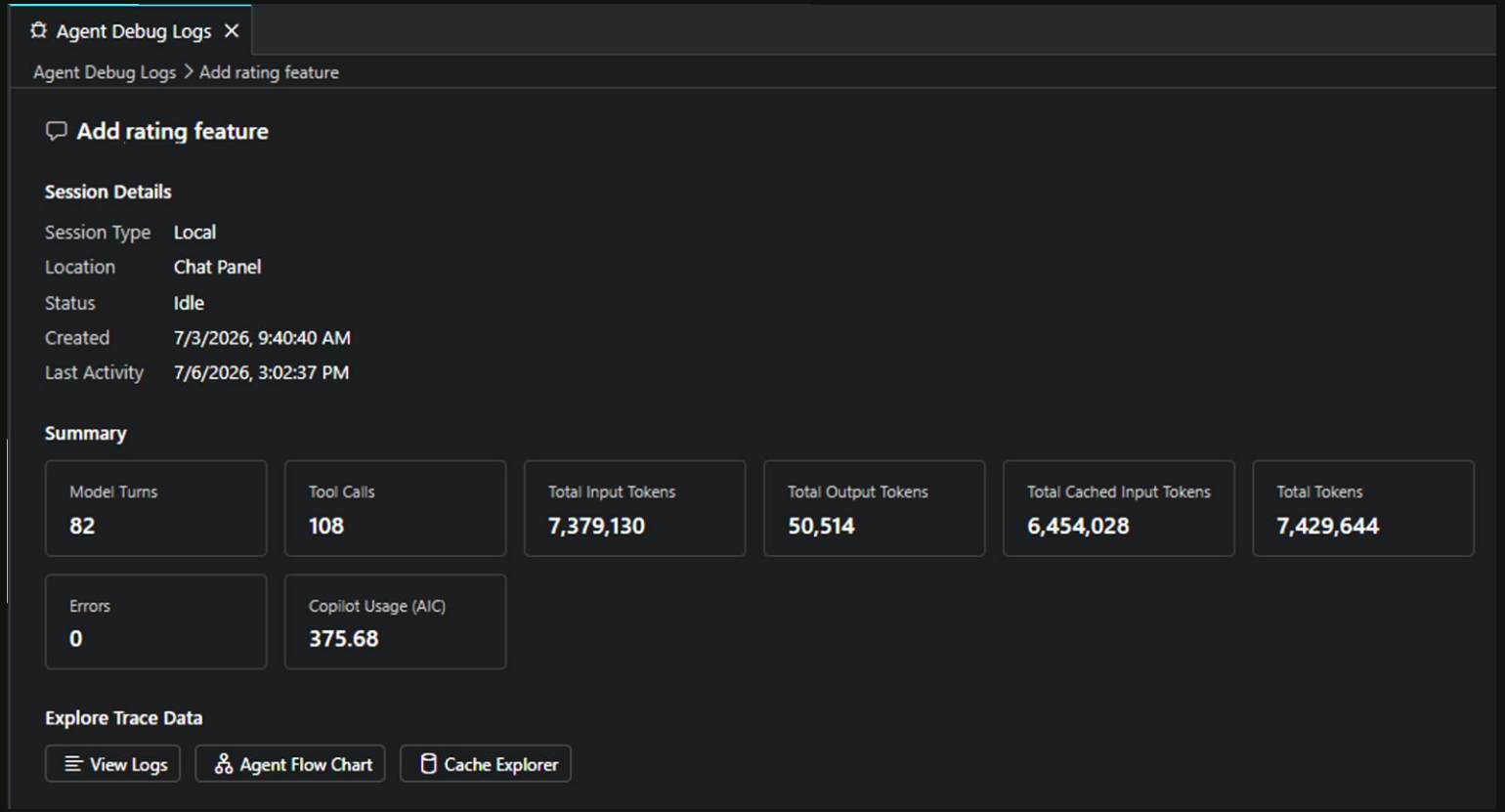
Illustrative: a 20-developer team on typical CLI usage. Numbers are not a commitment — run /usage on your own team for a real baseline.

SECTION 06

Three Tools You Can Use Today To Improve Your Usage



Agent Debug Log Summary Page

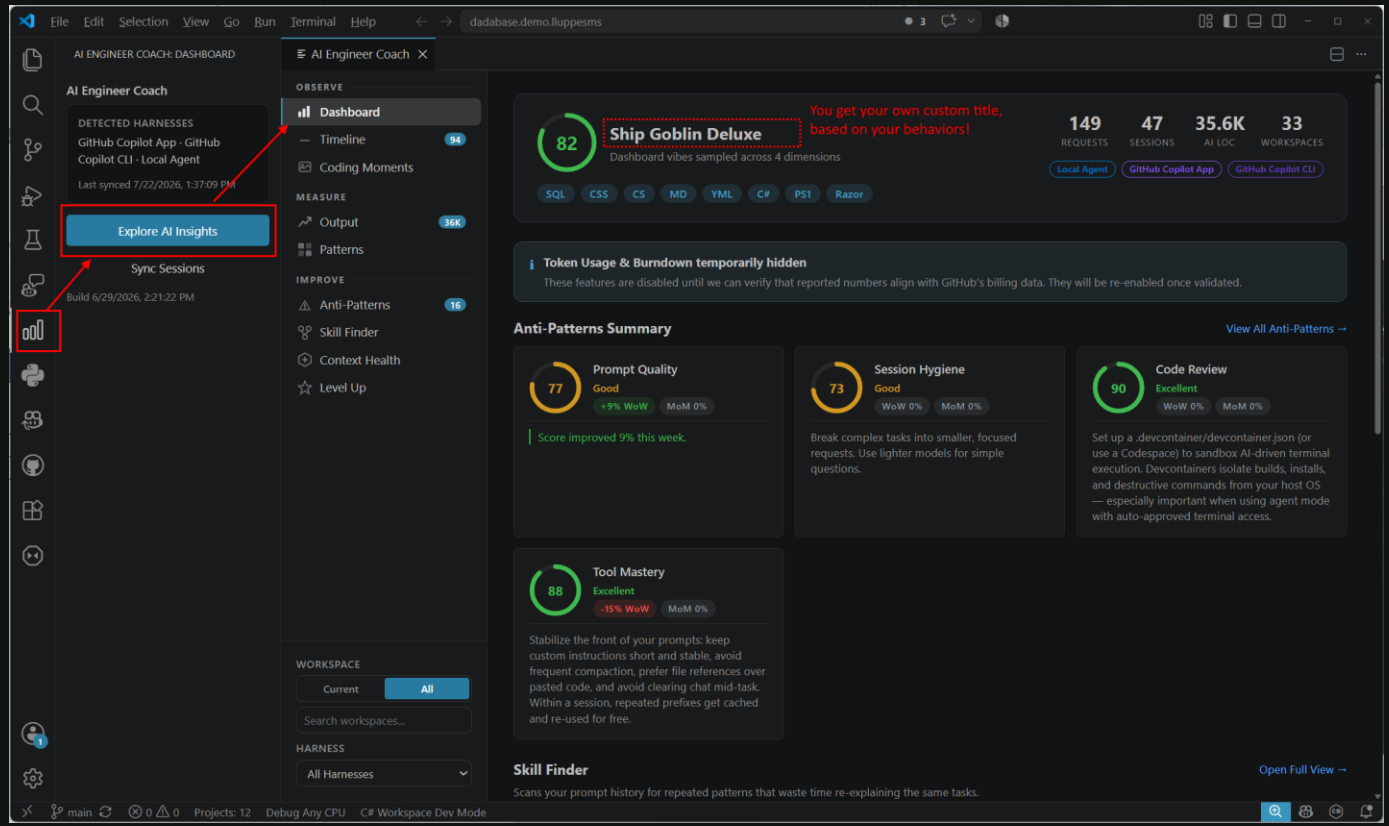


View Cost Tips in the Copilot CLI using Chronicle

```
❯ /chronicle
  /chronicle standup  Get a report on your work from the last day
  /chronicle search  Search all session content by keyword or topic
  /chronicle tips  Get personalized tips based on usage patterns
  /chronicle cost-tips  Get personalized tips to reduce token usage and cost
  /chronicle improve  Suggest improvements to copilot-instructions.md

/chronicle [standup|search|tips|cost-tips|improve|reindex]
```

```
↓ /chronicle cost-tips                                                    07:42

 Top cost drivers

 1. A few sessions dominate spend.
    Evidence: top 3 sessions used 113,175,843 / 187,276,083 tokens (~60.4%); top 5 used ~78.7%;
    top 10 used ~96.2%. Precision: Exact usage.
 2. Very large context windows in long-running chats.
    Evidence: largest sessions had massive input vs output (e.g., 54,323,667 in / 254,859 out;
    33,392,022 in / 114,291 out), showing repeated re-sending of large context. Precision:
    Exact usage.
 3. VS Code Chat is the primary cost surface.
    Evidence: VS Code Chat = 153,989,199 tokens (82.23%); Copilot CLI = 33,286,884 (17.77%).
    Precision: Exact usage.
 4. Oversized pasted content (especially terminal dumps) is recurring.
    Evidence: individual user messages up to 60,144 , 45,069 , 44,837 , 40,558 chars, many
    containing terminal output blocks. Precision: Proxy.
 5. High-cost model concentration.
    Evidence: gpt-5.3-codex accounted for 83,761,443 tokens (44.73%) overall in scope.
    Precision: Exact usage.

 Ranked recommendations

 1. Split long investigations into fresh chats more aggressively
    • Context: Costs spike when the same conversation carries too much prior context turn after
      turn.
    • Problem: Your top sessions were extremely large and context-heavy (top 3 = ~60% of total
      tokens, with very high input/output ratios).
    • Recommended action: After each milestone (e.g., diagnosis complete, implementation complete),

C:\lyle\Dadabase\dadabase.cli.lluppesms [ feature/cli-ratings]          Session: 439 AIC used
```

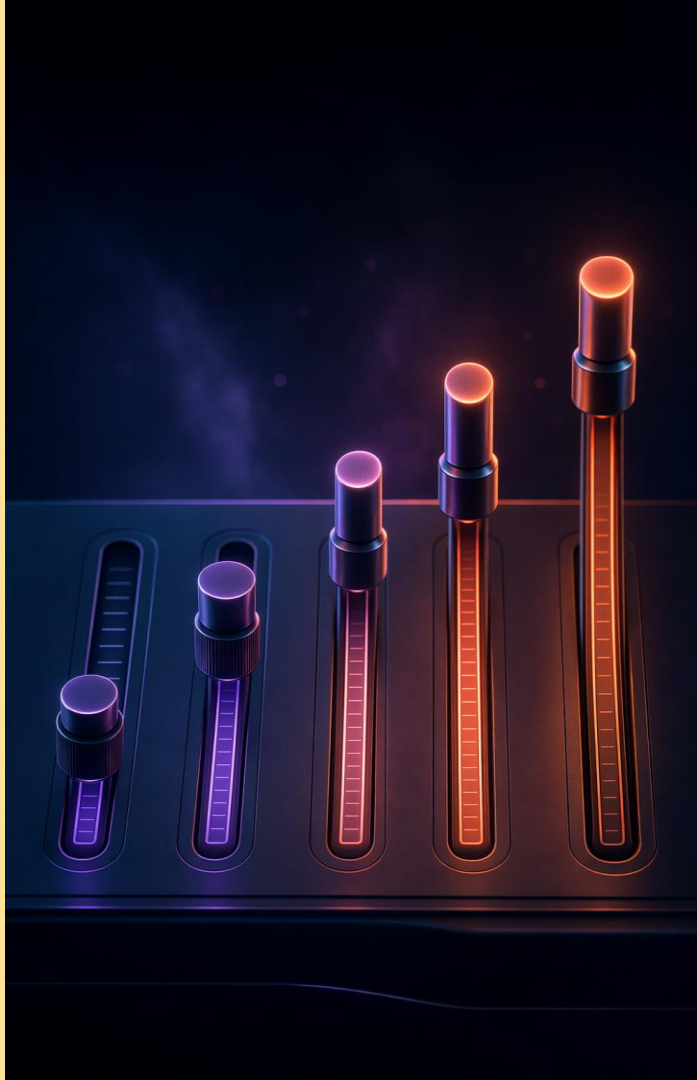
Tip: Install the VS Code Extension: “AI Engineer Coach”



https://github.com/microsoft/AI-Engineering-Coach

SECTION 07

Call to Action



GO DEEPER

References & further reading

Optimizing GitHub Copilot Cost in the Usage-Based Billing Era

Azure Architecture Blog – July 7, 2026

techcommunity.microsoft.com/blog/azurearchitectureblog

Improving token efficiency for Copilot in VS Code

VS Code engineering blog · June 17, 2026

code.visualstudio.com/blogs

Improve agent quality & token optimization

GitHub Support — product guide

support.github.com/product-guides

Monitor AI coding agents with Grafana

Microsoft Learn — Managed Grafana + OTel

learn.microsoft.com/azure/managed-grafana

Token Economy: Copilot Chat & agents under UBB

Community playbook (aj-enns) · May 2026

github.com/aj-enns/token-economy

Copilot models, plans & pricing

GitHub Docs — living source of truth

docs.github.com/en/copilot

Improving token efficiency in Agentic Workflows

GitHub blog · May 7, 2026

github.blog/ai-and-ml/github-copilot

Tokenomics is the new headcount

Jared Spataro (Microsoft CMO of AI at Work) Blog posted June 2026

www.microsoft.com/en-us/worklab/aiwork-tokenomics-is-the-new-headcount-and-four-more-trends-to-watch

IN SUMMARY

Top ten things you can start doing today

01	Choose the right model for the task: use Auto mode, escalate only when needed
02	Give clear, focused guidance in your prompts
03	Work in phases - fresh window each time: Research → Plan → Implement
04	Add deterministic guardrails: tests, linters, security scans
05	Keep a concise, human-written copilot-instructions.md
06	Start a new session per task: /clear or /new between unrelated work
07	Reference files with @path/to/file, not whole directories
08	End every session with /usage: know what it cost
09	Open with /plan for anything beyond a one-line change
10	Start small – start now: run /usage after your next three sessions. You'll know where to begin.

- *If you remember one line: write as little context as required, and as much as necessary.*

Questions?

Thank you — now go make every token count.

Start small: run /usage at the end of your next three sessions. You'll know your baseline and where to begin.